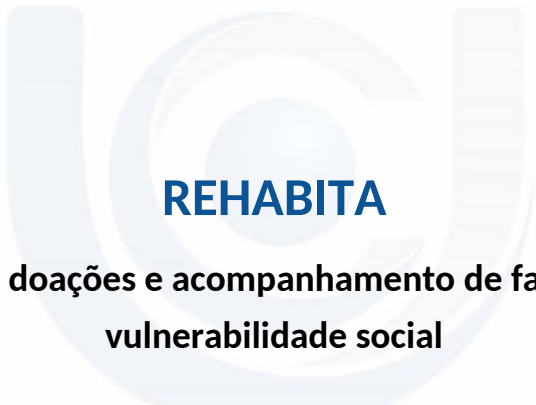


UNIVERSIDADE CESUMAR – UNICESUMAR

AMANDA RODRIGUES NEVES

JULIA OLIVEIRA DOS SANTOS

JULIA YUMI HASHIMOTO



REHABITA

Sistema de gestão de doações e acompanhamento de famílias em situação de vulnerabilidade social

Atividade de Estudo Programada (AEP) – 4º semestre

1ª Entrega: Planejamento, Escopo e Estruturação (1º bimestre)

MARINGÁ – PR

2026

1. Descoberta

1.1 Como surgiu a ideia

A ideia do REHABITA surgiu observando o trabalho de instituições de Maringá que ajudam famílias em situação de vulnerabilidade, como paróquias, igrejas, pastorais, a APAE e projetos sociais de bairro. Essas instituições recebem doações o tempo todo (cesta básica, produtos de higiene, fraldas, roupas, cobertores) e repassam para famílias da comunidade que estão passando necessidade.

O que chamou a nossa atenção foi a forma como tudo isso é controlado. Na maioria dos casos o registro fica em caderno, em uma planilha feita por algum voluntário ou em grupo de WhatsApp. Funciona enquanto são poucas famílias e as mesmas pessoas cuidam de tudo, mas basta trocar o voluntário ou aumentar a procura (como acontece no inverno e no fim do ano) para a informação começar a se perder.

Nas conversas com pessoas que atuam nesse tipo de trabalho, algumas perguntas apareciam sempre e ninguém conseguia responder rápido: quantas cestas ainda temos guardadas? Essa família já recebeu este mês? Quem fez a última visita e o que ficou combinado? Quais famílias estão em situação mais grave agora? Foi a partir dessas perguntas que montamos o levantamento de dores e, depois, a lista de requisitos.

A proposta, então, é um sistema para a própria instituição organizar o ciclo completo: a doação que chega, o estoque, a família que é cadastrada e visitada e o que foi entregue para ela. Quem usa o sistema é a equipe da instituição, ou seja, a coordenação e as pessoas responsáveis pelas visitas.

1.2 Partes interessadas

Identificamos quatro grupos envolvidos. Só os dois primeiros usam o sistema diretamente. Os outros dois não acessam a aplicação, mas são afetados pelo resultado dela.

Coordenação da instituição (usa o sistema): é quem cuida da arrecadação e da distribuição nas paróquias, igrejas, APAE e projetos sociais. Precisa saber o que tem em estoque, o que foi entregue e para quem, e quais famílias precisam de prioridade. Também é quem cadastra os usuários, os itens e os bairros.

Assistentes sociais, agentes de pastoral e voluntários de visita (usam o sistema): fazem o cadastro das famílias e as visitas. Precisam ter o histórico da família em mãos para não começar do zero a cada atendimento.

Famílias assistidas (não usam o sistema): são as mais afetadas pelo resultado. Com o cadastro organizado, a família não precisa repetir a mesma história a cada visita e tem menos chance de ficar esquecida quando a situação piora.

Doadores (não usam o sistema): pessoas, empresas e campanhas de arrecadação (de escolas, rádios, comércio) que entregam as doações. Eles não acessam o REHABITA, quem registra a doação é a própria instituição. Mesmo assim o doador é beneficiado, porque a instituição passa a conseguir mostrar quanto recebeu e quanto já foi repassado, o que ajuda a manter a confiança de quem doa.

1.3 Dores identificadas e origem dos problemas

O quadro abaixo resume os problemas que mais apareceram, por que eles acontecem e qual parte do sistema responde a cada um. Os códigos R e RN são os requisitos e as regras de negócio da seção 3.

Quadro 1 – Dores, origens e resposta do sistema

O problema (dor)	Por que acontece	O que o REHABITA faz
A informação da família se perde ou precisa ser perguntada de novo a cada visita.	Os dados ficam em cadernos, planilhas e conversas de WhatsApp, e cada voluntário tem a sua versão.	Cadastro único da família e dos membros (R1, R2).
Algumas famílias recebem duas vezes no mesmo mês e outras ficam sem nada.	Não existe um lugar onde dê para ver o que cada família já recebeu e quando.	Histórico de entregas e aviso de entrega repetida (R6, RN4).
Casos urgentes esperam na mesma fila que os outros.	Não há um jeito combinado de marcar urgência nem de lembrar quem precisa de retorno.	Nível de urgência, data de retorno e lista de prioridade (R3, R9, RN7).
A coordenação não sabe quanto tem em estoque nem quando algo vai acabar.	As doações chegam por vários lados e são guardadas sem anotar entrada e saída.	Registro das doações recebidas e saldo por item (R4, R5, RN3).
É difícil prestar contas para quem doou e para a própria instituição.	Nada liga o que entrou com o que foi entregue.	Doações e entregas ficam gravadas com data e responsável (R5, R6).
Não dá para saber quais bairros concentram mais famílias atendidas.	O endereço é anotado de qualquer jeito, então não dá para agrupar.	Bairro escolhido de uma lista e consulta por bairro (R1, R9).

2. Concepção do projeto e alinhamento às ODS

2.1 O que é o REHABITA

O REHABITA é um sistema desktop para instituições que distribuem doações a famílias em situação de vulnerabilidade social. Ele junta em um lugar só o cadastro das famílias, o registro das visitas, o controle das doações recebidas e o histórico do que foi entregue para cada família.

2.2 Como o sistema funciona no dia a dia

O sistema segue o mesmo caminho que a doação já faz hoje dentro da instituição, só que agora tudo fica registrado:

1. Chega uma doação. A coordenação lança o que entrou (itens e quantidades) e quem doou, se o doador quiser se identificar. As quantidades somam no estoque.
2. Uma família é indicada ou procura a instituição. É feito o cadastro do responsável, do endereço e dos membros, e a família entra com status Novo.
3. Alguém da equipe faz a visita e registra o atendimento, marcando o nível de urgência e quando deve voltar. A família passa para Em acompanhamento.
4. Se houver entrega nesse atendimento, os itens são lançados ali mesmo. O sistema confere o saldo, avisa se a família já recebeu o mesmo item há pouco tempo e desconta do estoque.
5. A coordenação consulta a lista de prioridade, o saldo dos itens e as famílias por bairro para decidir as próximas visitas e quando fazer uma nova campanha.

2.3 ODS atendidas

O projeto se relaciona principalmente com três Objetivos de Desenvolvimento Sustentável da ONU. Para não ficar só no discurso, ligamos cada ODS a uma meta específica e às funcionalidades que atendem essa meta.

ODS 1 – Erradicação da Pobreza (meta 1.3). A meta trata de proteção social que realmente alcance as pessoas pobres e vulneráveis. As instituições de bairro fazem parte dessa rede e muitas vezes são o primeiro lugar que a família procura. O REHABITA contribui porque cada família cadastrada tem status, histórico de visitas e data de retorno (R3, R7), então fica visível quando alguém está há muito tempo sem atendimento.

ODS 2 – Fome Zero e Agricultura Sustentável (meta 2.1). A maior parte do que essas instituições distribuem é alimento, principalmente a cesta básica, e a meta 2.1 fala justamente em garantir acesso a alimento para quem está em situação de vulnerabilidade. Com o controle de estoque e de entregas (R4, R5 e R6), a coordenação percebe que a cesta está acabando antes de faltar e consegue organizar uma campanha a tempo.

ODS 10 – Redução das Desigualdades (meta 10.2). A meta fala em inclusão social de todas as pessoas. No nosso caso isso aparece de um jeito bem prático: a lista de prioridade e o aviso de entrega repetida (RN4 e RN7) fazem a ajuda ser distribuída por critério, ou seja, pela urgência e pelo tempo sem receber, e não por quem chegou primeiro ou conhece alguém da equipe.

A consulta por bairro (R9) também mostra onde estão concentradas as famílias atendidas, o que conversa com a ODS 11 (Cidades e Comunidades Sustentáveis), mas consideramos essa uma relação secundária.

Sabemos que um sistema sozinho não resolve a pobreza nem a fome. O que ele pode fazer é tirar o trabalho dessas instituições do improviso, para que o pouco que chega seja bem aproveitado e vá para quem mais precisa.

3. Escopo: lista de requisitos

3.1 Requisitos

Os requisitos abaixo definem o que o sistema vai fazer na 2ª entrega, e cada um veio de uma dor da seção 1.3. A entidade principal, sobre a qual será feito o CRUD completo, é a Família (R1).

Quadro 2 – Lista de requisitos

Nº	Requisito
R1	O sistema deve permitir o cadastro de famílias assistidas, com os dados do responsável (nome completo, CPF, data de nascimento, telefone, escolaridade e situação de trabalho), o endereço (CEP, rua, número, complemento e bairro), o tipo de moradia e a faixa de renda, além de consultar, editar e excluir esse cadastro.
R2	O sistema deve permitir o cadastro dos membros de cada família, com nome completo, data de nascimento, grau de parentesco com o responsável, telefone e CPF (esses dois opcionais), calculando automaticamente quantas pessoas moram na casa.
R3	O sistema deve permitir o registro dos atendimentos e visitas feitos a cada família, com data da visita, nível de urgência (baixa, média ou alta), usuário que atendeu, observações e data prevista para o retorno.
R4	O sistema deve permitir o cadastro dos itens distribuídos pela instituição (ex.: cesta básica, kit de higiene, fralda, cobertor), com nome, categoria (alimento, higiene, vestuário ou outros) e unidade de medida, mostrando o saldo atual em estoque de cada item.
R5	O sistema deve permitir o registro das doações recebidas, com data, doador (pessoa física, empresa, instituição ou doação anônima) e os itens com suas quantidades, somando essas quantidades ao estoque.
R6	O sistema deve permitir o registro dos itens entregues à família em cada atendimento, com a quantidade, descontando do estoque e mantendo o histórico de tudo o que a família já recebeu.
R7	O sistema deve permitir a alteração do status de acompanhamento da família (Novo, Em acompanhamento ou Inativo), guardando a data, o usuário e o motivo de cada mudança.
R8	O sistema deve permitir o cadastro de usuários (nome, CPF, telefone, e-mail, senha e perfil) e o login com e-mail e senha, com dois perfis: coordenador, que acessa tudo e é o único que cadastra usuários, itens e bairros, e assistente social, que trabalha com famílias, atendimentos, entregas, doações e consultas.
R9	O sistema deve permitir consultar as famílias por bairro, status e nível de urgência, gerar a lista de prioridade de atendimento e o relatório de saldo dos itens em estoque.

3.2 Regras de negócio

Durante o levantamento apareceram algumas regras que o sistema precisa respeitar. No código elas ficam na camada service e, quando dá, também no banco de dados, como uma segunda proteção.

Quadro 3 – Regras de negócio

Código	Regra	Requisito
RN1	Não pode existir duas famílias com o mesmo CPF de responsável. Se o CPF já estiver cadastrado, o sistema mostra a família que já existe em vez de criar outra.	R1
RN2	A quantidade de pessoas da família não é digitada: é o responsável mais os membros cadastrados.	R2
RN3	O saldo de um item só muda por doação recebida (entrada) ou por entrega (saída), nunca é editado à mão. A entrega só é gravada se houver saldo suficiente, e o registro da entrega e a baixa no estoque são salvos juntos: se um falhar, nenhum dos dois é salvo.	R4, R5, R6
RN4	Se a família já recebeu o mesmo item nos últimos 30 dias (a cesta básica costuma ser mensal), o sistema avisa. A entrega só continua se o atendimento estiver marcado com urgência alta.	R6
RN5	Toda família começa com status Novo e passa para Em acompanhamento no primeiro atendimento. Qualquer mudança manual de status exige um motivo e fica gravada no histórico.	R3, R7
RN6	Família que já tem atendimento registrado não pode ser excluída, só inativada, para não apagar o histórico de entregas.	R1, R7
RN7	A lista de prioridade mostra as famílias que não estão inativas, ordenadas pela urgência do último atendimento e, em caso de empate, por quem está há mais tempo sem receber entrega. Família com status Novo entra como urgência média até receber a primeira visita.	R9

3.3 O que fica fora desta versão

Para o projeto caber no tempo do 2º bimestre, deixamos algumas coisas de fora de propósito: acesso de doadores ou das próprias famílias ao sistema (quem usa é só a equipe da instituição); versão web ou aplicativo de celular; várias instituições usando o mesmo banco (cada instituição tem a sua instalação); integração com o CadÚnico ou outros sistemas do governo; e controle de validade dos alimentos. Esses pontos ficam anotados como melhorias para uma próxima versão.

4. Cronograma de execução

O cronograma abaixo é o backlog do 2º bimestre, dividido por semana. A ordem segue a dependência entre as partes: primeiro o ambiente e o banco, depois o cadastro de famílias (entidade principal), em seguida atendimentos e estoque e, por último, consultas, testes e documentação. Cada integrante tem tarefas em todas as fases, para que o histórico do GitHub mostre a participação de todas. A mesma tabela está no README do repositório.

Quadro 4 – Cronograma e responsáveis

Semana	Período	Atividade	Requisito	Responsável
1	14/09 a 20/09	Criar o projeto Java com Maven, os pacotes (model, view, controller, service, dao, util) e a classe ConexaoBanco lendo o arquivo db.properties	–	Julia O.
		Instalar o SQL Server, habilitar TCP/IP na porta 1433, criar o login reabilita_app e testar os scripts da pasta /database	–	Amanda
2	21/09 a 27/09	Implementar as classes Pessoa, Responsavel, Membro e Usuario e os enums do pacote model	R1, R2, R8	Julia Y.
		Criar a interface Dao<T> e o BairroDao, testando leitura e gravação no banco	R1	Amanda
3	28/09 a 04/10	Implementar Familia e FamiliaDao com o CRUD completo (inserir, buscar, listar, atualizar e excluir) e a regra do CPF único	R1	Amanda
		UsuarioDao, cadastro de usuários e tela de login com senha em hash e controle de perfil	R8	Julia O.
4	05/10 a 11/10	Telas de listagem e cadastro de famílias ligadas ao FamiliaController	R1	Julia Y.
		Cadastro de membros (MembroDao e aba na tela da família) com cálculo da quantidade de pessoas	R2	Julia O.
5	12/10 a 18/10	Atendimento, AtendimentoDao e tela de registro de visita	R3	Julia Y.
		Status da família com histórico e regra de excluir ou inativar	R7	Amanda
6	19/10 a 25/10	ItemDoacao, MovimentacaoItem e ItemRecebido, com cadastro de itens e saldo	R4	Amanda
		Doador, Doacao e tela de doações recebidas somando no estoque	R5	Julia O.
7	26/10 a 01/11	Entregas no atendimento (ItemEntregue e EntregaService) com baixa de estoque em transação e aviso de entrega repetida	R6	Julia Y.
		Testar as regras de estoque (saldo insuficiente, entrega repetida, urgência alta)	R6	Amanda

Semana	Período	Atividade	Requisito	Responsável
8	02/11 a 08/11	Consultas de famílias por bairro, status e urgência e lista de prioridade	R9	Julia O.
		Relatório de saldo em estoque e menu principal de acordo com o perfil	R8, R9	Julia Y.
9	09/11 a 15/11	Teste geral do sistema com dados de exemplo e correção de erros	Todos	Amanda, Julia O. e Julia Y.
		Rodar o projeto do zero em outra máquina (Windows e Linux com Docker) seguindo o README	-	Julia Y.
10	16/11 a 22/11	Completar o README com o passo a passo de execução e atualizar os diagramas se algo mudou	-	Julia O.
		Revisão final do código (nomes em CamelCase, uso de @Override, código sem uso) e versão v1.0 no GitHub	-	Amanda

Obs.: Julia O. = Julia Oliveira dos Santos; Julia Y. = Julia Yumi Hashimoto.

5. Justificativa técnica e arquitetural

Resumindo: o REHABITA vai ser feito em Java 21, com banco de dados SQL Server, organizado em camadas no padrão MVC com DAO e com telas em Swing. Nas próximas seções explicamos o motivo de cada escolha, sempre ligando com os requisitos da seção 3.

5.1 Linguagem: Java

Escolhemos Java porque o domínio do REHABITA tem vários conceitos que se encaixam bem em orientação a objetos. Responsável, membro e usuário são pessoas com dados em comum; entrada e saída de estoque são movimentações que mexem no saldo de formas opostas. Com herança e polimorfismo isso vira código reaproveitado, em vez de vários if espalhados pelo sistema (detalhes na seção 6).

A tipagem forte também pesou. Status da família, nível de urgência, perfil e categoria de item vão ser enums, então não existe o risco de gravar "urgente" em um lugar e "ALTA" em outro, o que quebraria a lista de prioridade (R9). As datas usam a API java.time (LocalDate), que facilita as contas que o sistema precisa fazer, como saber se a família recebeu o mesmo item nos últimos 30 dias (RN4) ou há quantos dias está sem entrega (RN7).

Também contaram alguns motivos práticos. O JDBC já faz parte do Java e a Microsoft mantém um driver oficial para o SQL Server (mssql-jdbc). O Swing vem junto com o JDK, então a interface não precisa de biblioteca extra. A senha pode ser guardada com hash PBKDF2 usando só classes do próprio Java. E o mesmo programa roda em Windows, Linux e macOS por causa da JVM. Vamos usar o JDK 21, que é uma versão LTS (com suporte de longo prazo), e o Java tem muito material de estudo e uma comunidade grande, o que ajuda quando a gente travar em algum erro.

www.unicesumar.edu.br

5.2 Banco de dados: SQL Server

Os dados do REHABITA são muito ligados entre si: uma família tem vários membros, vários atendimentos e um histórico de status; cada atendimento pode ter vários itens entregues; cada doação tem vários itens recebidos. Por isso a escolha foi um banco relacional, em que as chaves estrangeiras impedem, por exemplo, uma entrega apontando para um item que não existe.

No começo pensamos em usar MySQL, mas decidimos pelo SQL Server porque ele atende bem alguns pontos que usamos diretamente nas regras do sistema:

Restrições CHECK: os campos de status, urgência, perfil e categoria só aceitam os mesmos valores dos enums Java, e o saldo em estoque não pode ficar negativo (RN3). Mesmo que escape algum erro no código, o banco recusa o dado errado.

Índice único filtrado: o CPF do membro é opcional, porque criança muitas vezes ainda não tem, mas quando é preenchido não pode repetir. No SQL Server isso é um índice UNIQUE com a condição WHERE cpf IS NOT NULL.

Regras de exclusão nas chaves estrangeiras: membros e histórico de status usam ON DELETE CASCADE, porque não existem sem a família. Já a ligação entre família e atendimento não tem cascade, então o próprio banco impede excluir uma família que tem atendimentos (RN6).

Transações: o registro da entrega e a baixa no estoque acontecem na mesma transação (no Java, com setAutoCommit(false), commit e rollback). Se faltar saldo no meio do processo, nada é gravado (RN3).

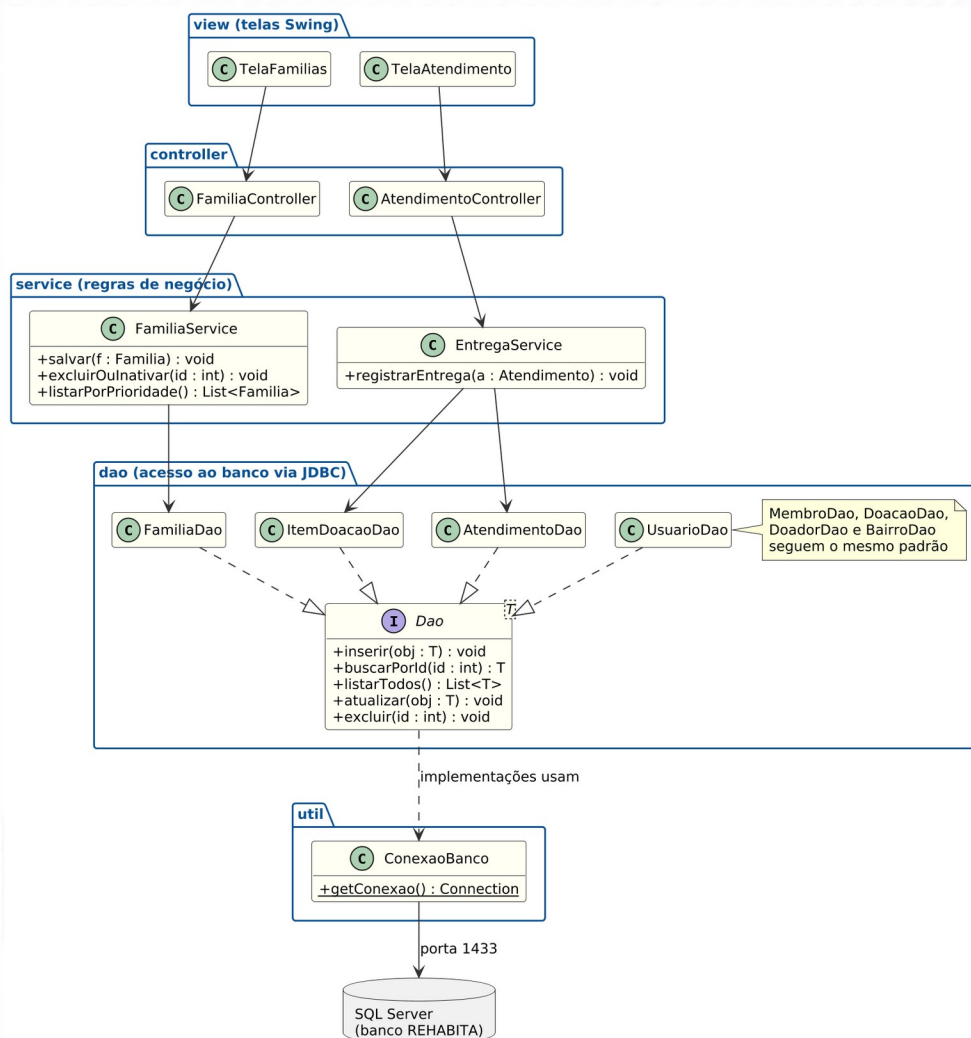
Para o desenvolvimento vamos usar a edição Developer ou a Express, que são gratuitas. O SQL Server roda como instância local com o protocolo TCP/IP habilitado no SQL Server Configuration Manager, na porta padrão 1433 (liberada no firewall), e com autenticação mista, já que o sistema conecta com um login SQL. A aplicação não usa o usuário sa: criamos o login rehabita_app, que só tem permissão para ler e gravar dados (db_datareader e db_datawriter). Como o banco guarda CPF, endereço e renda das famílias, que são dados pessoais protegidos pela LGPD, faz sentido limitar esse acesso desde o começo.

Pensando na compatibilidade de ambientes vista em Sistemas Operacionais: no Windows o SQL Server é instalado normalmente, no Linux ele também tem instalação nativa e, em qualquer sistema (inclusive macOS), dá para subir a imagem oficial no Docker. Os dados de conexão ficam no arquivo db.properties, fora do código, então cada integrante (e o professor) troca servidor, porta, usuário e senha sem precisar recompilar. Para administrar o banco e rodar os scripts usamos o SQL Server Management Studio (SSMS) no Windows ou a extensão SQL Server (mssql) do VS Code nos outros sistemas.

5.3 Arquitetura: MVC em camadas com DAO

O código vai ser separado em camadas, e cada camada tem um trabalho só: a tela não sabe SQL e o DAO não sabe regra de negócio. A figura 1 mostra como as camadas conversam, usando o cadastro de famílias e o registro de entregas como exemplo.

Figura 1 - Organização do código em camadas



Fonte: elaborado pelo grupo (2026).

Quadro 5 – Pacotes do projeto

Pacote	Responsabilidade	Exemplos
model	Classes do domínio e enums (a parte de POO da seção 6).	Familia, Membro, ItemEntregue, StatusFamilia
view	Telas em Swing. Só mostram os dados e capturam o que o usuário digitou.	TelaLogin, TelaFamilias, TelaAtendimento
controller	Recebe a ação da tela, chama a service e devolve a mensagem de sucesso ou de erro.	FamiliaController, AtendimentoController
service	Regras de negócio RN1 a RN7 e controle das transações.	FamiliaService, EntregaService
dao	Comandos SQL via JDBC com PreparedStatement. Todo DAO implementa a interface Dao<T>.	FamiliaDao, ItemDoacaoDao, UsuarioDao
util	Recursos usados por várias camadas.	ConexaoBanco, ValidadorCpf, SenhaUtil

Um exemplo de como isso atende o escopo: quando a assistente social registra uma entrega (R6), a TelaAtendimento chama o AtendimentoController, que chama o EntregaService. É na service que ficam as regras, como conferir o saldo (RN3) e verificar entrega repetida (RN4). Só depois disso ela pede para o AtendimentoDao e o ItemDoacaoDao gravarem, os dois usando a mesma conexão e a mesma transação.

O padrão DAO também ajuda no CRUD da entidade principal: o FamiliaDao implementa os cinco métodos da interface Dao<T> (inserir, buscarPorId, listarTodos, atualizar e excluir) com @Override, e os outros DAOs seguem o mesmo contrato. Todas as consultas usam PreparedStatement com parâmetros, o que evita SQL injection nos campos digitados nas telas. E se um dia o sistema virar web, as camadas model, service e dao continuam as mesmas; muda só a view.

5.4 Interface

A interface vai ser feita em Swing, que já vem no JDK. O objetivo não é ter telas sofisticadas, e sim telas simples para quem não tem muita familiaridade com computador, o que é comum entre voluntários. Por isso decidimos usar listas em tabela (JTable) com os filtros de bairro, status e urgência logo no topo; caixas de seleção para os campos com valores fixos (status, urgência, categoria e perfil), para ninguém digitar errado; campos obrigatórios marcados e mensagens de erro claras, como "Saldo insuficiente: restam 3 cestas básicas"; e um menu principal que muda de acordo com o perfil, então a assistente social nem vê as opções que são só do coordenador (R8).

As telas previstas são: login, menu principal, famílias (lista e cadastro, com abas de membros e histórico), atendimento com entregas, doações recebidas, itens e estoque, usuários e consultas.

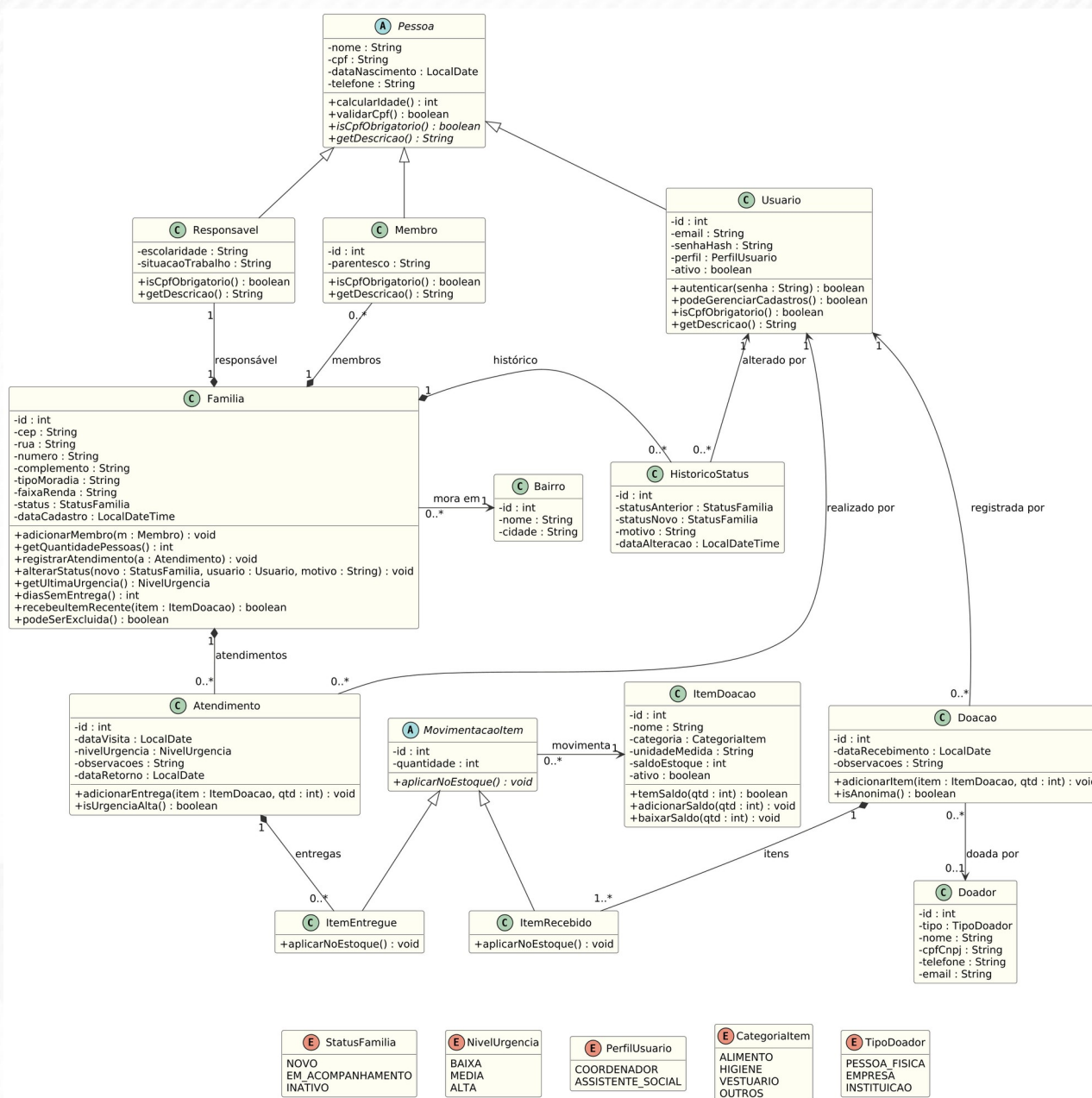
5.5 Ferramentas de apoio

Git e GitHub para o versionamento; Maven para organizar o projeto e baixar o driver JDBC; IntelliJ IDEA Community ou VS Code como editor (cada integrante usa o que preferir, porque o projeto Maven abre nos dois); SSMS ou VS Code para o banco; e PlantUML para os diagramas. Os diagramas ficam salvos como texto (arquivos .puml) na pasta /docs, então o histórico do Git também mostra o que mudou no modelo ao longo do projeto.

6. Diagrama de classes

A figura 2 mostra as classes do pacote model. A classe central é Família, que reúne o responsável, os membros, o histórico de status e os atendimentos. A parte de estoque gira em torno de ItemDoacao, com as doações recebidas (Doacao) de um lado e as entregas feitas nos atendimentos do outro. Para o diagrama não ficar poluído, os atributos que são outras classes (por exemplo, a lista de membros da família) aparecem só como linhas de relacionamento, e os métodos get e set foram omitidos.

Figura 2 – Diagrama de classes do REHABITA



Fonte: elaborado pelo grupo (2026).

Herança. O modelo tem duas hierarquias. A primeira é a classe abstrata Pessoa, com nome, CPF, data de nascimento e telefone, da qual herdam Responsavel, Membro e Usuario. A segunda é a classe abstrata MovimentacaoItem, com a quantidade e o item movimentado, da qual herdam ItemRecebido (entrada por doação) e ItemEntregue (saída para a família).

Polimorfismo. Pessoa tem dois métodos abstratos que cada subclasse sobrescreve com @Override. O isCpfObrigatorio() é usado pelo validarCpf(): Responsavel e Usuario retornam true, e Membro retorna false, porque criança muitas vezes ainda não tem CPF. O getDescricao() monta o texto que aparece nas telas, por exemplo "Maria Souza (responsável, 42 anos)" ou "Pedro Souza (filho, 8 anos)". Em MovimentacaoItem está o caso mais importante, o aplicarNoEstoque(): em ItemRecebido ele soma a quantidade no saldo do item e em ItemEntregue ele confere se há saldo e

desconta. A service percorre uma lista de movimentações e chama aplicarNoEstoque() sem precisar saber de qual tipo é cada uma. Além das classes abstratas, a interface Dao<T> da figura 1 também é implementada com @Override por todos os DAOs.

Composição 1:N. Família é composta por Membro (0..*), HistoricoStatus (0..*) e Atendimento (0..*); Atendimento é composto por ItemEntregue (0..*); e Doacao é composta por ItemRecebido (1..*, porque não existe doação sem item). Usamos composição (losango preto) porque essas partes não existem sozinhas: um membro não faz sentido sem a família e um item entregue não existe fora do atendimento. Responsavel também é uma composição de Família, mas com multiplicidade 1:1.

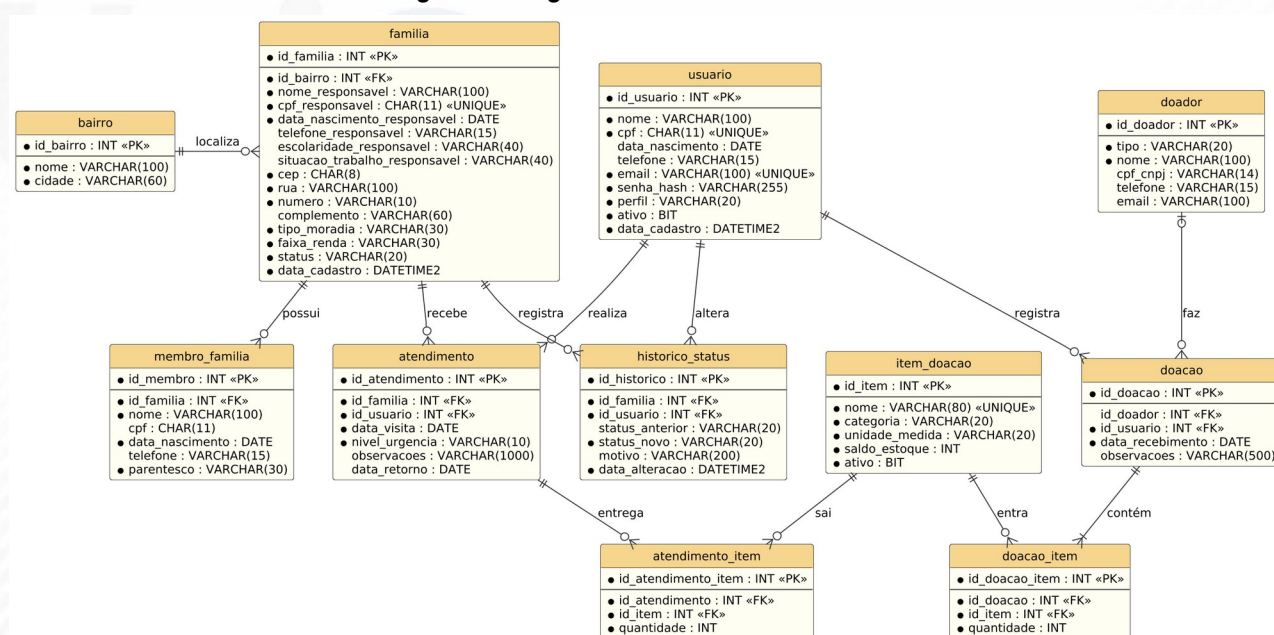
Associações. Família aponta para Bairro; Atendimento, HistoricoStatus e Doacao apontam para o Usuario que fez o registro; Doacao aponta para Doador com multiplicidade 0..1, porque a doação pode ser anônima; e toda MovimentacaoItem aponta para um ItemDoacao.

Enums. StatusFamília, NivelUrgencia, PerfilUsuario, CategoricalItem e TipoDoador guardam os valores fixos do sistema. Os mesmos valores estão nas restrições CHECK do banco.

7. Diagrama do banco de dados (DER)

O DER da figura 3 foi feito a partir do diagrama de classes, já com os tipos do SQL Server (INT com IDENTITY nas chaves primárias, VARCHAR, CHAR, DATE, DATETIME2 e BIT). Os campos marcados com asterisco são obrigatórios (NOT NULL). O script que cria essas tabelas está em /database/02_criar_tabelas.sql, com os mesmos nomes de tabelas e colunas do diagrama.

Figura 3 – Diagrama entidade-relacionamento



Fonte: elaborado pelo grupo (2026).

Como o banco relacional não tem herança, precisamos decidir como cada classe vira tabela. As classes abstratas (Pessoa e MovimentacaoItem) não viram tabela, e os atributos delas vão para a tabela de cada subclasse. O Responsavel, por ser 1:1 com a família, ficou dentro da própria tabela família (colunas terminadas em _responsavel), o que evita um JOIN a mais em toda consulta de família. O quadro 6 mostra a correspondência completa entre classes e tabelas.

Quadro 6 – Correspondência entre classes e tabelas

Classe (Java)	Tabela (SQL Server)	Observação
Pessoa (abstrata)	-	Atributos vão para as tabelas das subclasses.
Responsavel	familia	Colunas *_responsavel (composição 1:1).
Membro	membro_familia	FK id_familia com ON DELETE CASCADE; CPF opcional e único.
Usuario	usuario	Perfil com CHECK (COORDENADOR ou ASSISTENTE_SOCIAL).
Familia	familia	cpf_responsavel UNIQUE (RN1); status com CHECK.
Bairro	bairro	Nome e cidade formam uma chave única.
HistoricoStatus	historico_status	FK para familia (CASCADE) e para usuario.
Atendimento	atendimento	FK para familia sem cascade (RN6).
ItemDoacao	item_doacao	saldo_estoque com CHECK maior ou igual a zero (RN3).
MovimentacaoItem (abstrata)	-	id e quantidade vão para as tabelas das subclasses.
ItemRecebido	doacao_item	Entrada no estoque.
ItemEntregue	atendimento_item	Saída do estoque.
Doacao	doacao	id_doador aceita NULL (doação anônima).
Doador	doador	Tipo com CHECK (PESSOA_FISICA, EMPRESA, INSTITUICAO).

A quantidade de pessoas da família não tem coluna, ela é sempre calculada (RN2), para nunca ficar diferente do número de membros cadastrados. O mesmo vale para a urgência atual da família, que é sempre a do último atendimento.

8. Controle de versão e estrutura do repositório

O projeto está versionado no GitHub, em repositório público:

<https://github.com/juliaoliveirads/rehabita-aep-2026>

A estrutura de pastas já está criada de acordo com as camadas da seção 5.3. Como o Git não guarda pasta vazia, as pastas que só vão receber código no 2º bimestre têm um arquivo .gitkeep.

Quadro 7 – Estrutura do repositório

Pasta / arquivo	Conteúdo
/src/main/java/br/com/rehabita/	Pacotes model, view, controller, service, dao e util (o código entra na 2ª entrega).
/src/main/resources/	db.properties.example, modelo do arquivo de conexão com o banco.
/database/	01_criar_banco.sql, 02_criar_tabelas.sql e 03_dados_iniciais.sql.
/docs/	Este documento em PDF.
/docs/diagramas/	Diagrama de classes, arquitetura em camadas e DER, em PNG e em PlantUML (.puml).
README.md	Descrição, ODS, requisitos, regras de negócio, cronograma e como preparar o ambiente.
.gitignore	Ignora a pasta target/, arquivos de IDE e o db.properties (a senha do banco não vai para o GitHub).

Para o trabalho em equipe combinamos algumas regras simples. A branch main só recebe código que compila. Cada integrante trabalha em uma branch com o nome da tarefa (por exemplo, feature/cadastro-familia) e junta na main por pull request, que outra integrante revisa antes. Os commits são pequenos e com mensagem dizendo o que foi feito. Assim o histórico mostra a participação de cada uma, seguindo a coluna de responsáveis do cronograma. Ao final de cada entrega marcamos uma versão no GitHub: v0.1 na 1ª entrega e v1.0 na 2ª.

Referências

MICROSOFT. **Microsoft JDBC Driver for SQL Server**. Repositório oficial. Disponível em: <https://github.com/microsoft/mssql-jdbc>. Acesso em: 10 set. 2026.

NAÇÕES UNIDAS BRASIL. **Objetivos de Desenvolvimento Sustentável**. Disponível em: <https://brasil.un.org/pt-br/sdgs>. Acesso em: 10 set. 2026.

PLANTUML. **PlantUML: documentação**. Disponível em: <https://plantuml.com>. Acesso em: 10 set. 2026.

UNICESUMAR. **Guia da AEP: 4º semestre (2026.2)**. Maringá: Unicesumar, 2026. Material da disciplina.